You should be able to attempt and solve Problems 1 and 4 before the lecture; Problems 1 and 3 require you to have read the lecture notes or watched the lecture. The second part of Problem 2 (the probability recurrence relation) is difficult, but what is asked is mostly for you to read the proof from the lecture and see if you understand it (not come up with it on your own).

Problem 5 is close to the algorithm and analysis of the algorithm seen in class, but requires a couple non-trivial ideas. It is worth trying before the tutorial to see where the difficulty lies in adapting the proof; but don't be discouraged if you don't find the solution on your own.

Problem 6 is "fun", and conceptually interesting; it is on the "difficult" side.

Problem 7 is not necessarily hard, but will require you to go in detail in the proof of the algorithm seen in class to see what to change. It is good practice.

The problems marked with a $(\star)$ are on the difficult side for their level (Warm-up, Problem solving, Advanced). Those with $(\star\star)$ are very difficult (or time-consuming).

Warm-up

Problem 1. Explain why every undirected graph on n vertices has exactly $2^{n-1} - 1$ distinct cuts (not all minimum cuts).

Solution 1. Each cut is identified by a partition of V. To count the number of possible partitions, consider choosing the number of subsets of $\{1, \ldots, n\}$, which is 2^n. So, we can choose a subset $A \subset V$ then let the rest be $B = V \setminus A$ as one partition. Due to the symmetry of partition, $(A_1, B_1) = (A_2, B_2)$ if $A_1 = B_2$ and we need to ensure that it is nonempty: $A_1 = \varnothing$ or $A_1 = \{1, \ldots, n\}$ (B_1 will be empty). We have

$$\frac{2^n - 2}{2} = 2^{n-1} - 1.$$

Problem 2. Solve the two recurrence relations for the Karger–Stein algorithm (time and probability).

Solution 2. For the runtime,

$$T(n) = 2T\left(n/\sqrt{2}\right) + O(n^2).$$

We can see the pattern:

$$\begin{aligned} T(n) &= 4T\left(\frac{n}{2}\right) + 2c \cdot \left(\frac{n}{\sqrt{2}}\right)^2 + cn^2 \\ &= 4T\left(\frac{n}{2}\right) + cn^2 + cn^2 \\ &= 8T\left(\frac{n}{2\sqrt{2}}\right) + 4c\left(\frac{n}{2}\right)^2 + cn^2 + cn^2 \\ &= 8T\left(\frac{n}{2\sqrt{2}}\right) + 3cn^2 \\ &= \sum_{j=0}^{O\left(\log_{\sqrt{2}} n\right)} 2^j c \cdot \left(\frac{n}{\left(\sqrt{2}\right)^j}\right)^2 = O(n^2 \log n). \end{aligned}$$

For the probability recurrence, see the proof from the lecture notes.

Problem 3. Analyse/describe what would happen to the time complexity and success probability if we only did 1 run (instead of 2) for the Karger–Stein algorithm. What if we did 3 runs instead of 2?

Solution 3. If we only do 1 run, it is the same as first one except you are checking every other step.

If we do 3 runs, we get difference recurrence equations: denote by T' the time complexity of the 3-run version c' denote the time to run 3 Modified Karger:

$$T'(n) = 3T'\left(n/\sqrt{2}\right) + c'n^2.$$

Solving this, we get

$$\begin{aligned}
T(n) &= 9T\left(\frac{n}{2}\right) + 3c' \cdot \left(\frac{n}{\sqrt{2}}\right)^2 + c'n^2 \\
&= 9T\left(\frac{n}{2}\right) + \frac{3}{2}c'n^2 + c'n^2 \\
&= 27T\left(\frac{n}{2\sqrt{2}}\right) + 9c'\left(\frac{n}{2}\right)^2 + \frac{3}{2}c'n^2 + c'n^2. \\
&= \sum_{k=1}^{t=O(\log_2 n)} \left(\frac{3}{2}\right)^{k-1} \cdot c'n^2 \\
&= c'n^2 \sum_{k=1}^{t=O(\log_2 n)} \left(\frac{3}{2}\right)^{k-1} \\
&= c'n^2 \frac{1\left(1-\left(\frac{3}{2}\right)^t\right)}{1-\frac{3}{2}} \\
&= 2c'n^2\left(\left(\frac{3}{2}\right)^{\log_2(O(n^2))} - 1\right) \\
&= 2c'n^2\left(\left(\frac{3}{2}\right)^{\log_{3/2}(O(n^2))\cdot\log_2(3/2)} - 1\right) \\
&= 2c'n^2((O(n^2))^{\log_2(3/2)} - 1) \\
&= O(n^{2\log_2 3}).
\end{aligned}$$

A slightly larger runtime if we use the same threshold.

Problem solving

Problem 4. Show how to, given as input a (multi)graph $G = (V, E)$ in either the adjacency matrix or adjacency list representation, to sample an edge uniformly at random in time $O(n)$, where $n = |V|$.

Solution 4. Suppose the data structure encodes the degree of any vertex, i.e., query to degree of any vertex V_i takes $O(1)$ operation. Query every vertex's degree and put into an array. We can then do the following:

1. Sample with probability $\Pr[V_i] = \frac{D_i}{\sum_{i=1}^n D_i}$, where D_i denote the degree of V_i.
2. Given sampled V_i, sample one of its neighbouring edges with probability $\frac{1}{D_i}$.

Combined, each edge could be selected if the algorithm selects one of its end points, and then choosing it. Without loss of generality, let $e = (i, j)$:

$$\Pr[T = e] = \frac{D_i}{\sum_{i=1}^n D_i} \cdot \frac{1}{D_i} + \frac{D_j}{\sum_{i=1}^n D_i} \cdot \frac{1}{D_j} = \frac{2}{\sum_{i=1}^n D_i} = \frac{2}{|E|}.$$

Also there is a sampling algorithm in the adjacency matrix representation with expected runtime of $O(n)$, when $m \geq n-1$ via rejection sampling: sample coordinates u.a.r. and if it hits one of the edges in the graph, return that edge; otherwise, repeat. The probability to hit one edge each time is $\frac{m}{n^2}$ and so in expectation, the runtime is $\frac{n^2}{m} = O(n)$ as $m \geq n-1$ (the graph is connected).

Adjacency list representation: Keep list of each vertex's degree, and total number of edges m. Pick i in $[m]$ u.a.r. Given the degrees, we can find in $O(n)$ time where the i-th edge start from. We then, given that vertex, can find the right (i'-th) neighbor in $O(n)$ time.

Problem 5. $(\star)$ Consider the following generalisation of MIN-CUT:

> k-MIN-CUT: Given an (undirected) connected graph $G = (V, E)$ on n vertices and m edges and an integer $k \geq 2$, output a k-cut $(A_1, \ldots, A_k)$ (partition of V) *minimising* the number $c_k(A_1, \ldots, A_k)$ of edges between the different connected components $A_1, \ldots, A_k$.

a) Adapt (the basic version of) Karger's algorithm to solve this problem.

b) Analyse the success probability and running time.

c) Provide a bound on the maximum number of k-Min-Cuts a graph can have.

Solution 5. We hereafter treat k as a constant (otherwise the running time, which is of the form $n^{O(k)}$, will not be polynomial in n).

a) Contract until $3k$ vertices are left. Then we can do a brute force (which takes $O(k^{3k})$ time – constant time for k constant.)

b) Denote $C = (A_1, A_2, \ldots, A_k)$ the minimum cut. Similarly, we look at the iterative process and denote the contracted graph at each step i, (V_i, E_i), for $i = 0, \ldots, n-k$. For each V_i, we can partition V_i into $L = \left\lceil \frac{|V_i|}{k} \right\rceil$ sets of size at most k: denote them by $S_1, \ldots, S_L$ (where $S_1, \ldots, S_{L-1}$ have size k; S_L could be less than that). For $j \leqslant L-1$, we claim that we have

$$\sum_{v \in S_j} \deg(v) \geq |C| \,.$$

To see why, suppose by contradiction that the sum of S_j's degree is less than $|C|$: then we can cut out every vertex in S_i and get a better cut than C, which is a contradiction. We then have

$$|E_i| = \frac{1}{2} \sum_{v \in V_i} \deg(v) = \frac{1}{2} \sum_{j=1}^{L} \sum_{v \in S_j} \deg(v) \geq \frac{1}{2} \left\lfloor \frac{|V_i|}{k} \right\rfloor |C| \geq \frac{1}{2} \left(\frac{|V_i|}{k} - 1 \right) |C|.$$

Therefore,

$$\frac{|C|}{|E_i|} \leqslant \frac{|C|}{\frac{1}{2}\left(\frac{|V_i|}{k} - 1\right)|C|} = \frac{2k}{|V_i| - k}.$$

As a result, the probability of getting the cut C (the algorithm will choose edges until $n-k-1$),

$$\begin{aligned}
\Pr[C \text{ is preserved}] &= \prod_{i=0}^{n-3k-1} \left(1 - \frac{|C|}{|E_i|}\right) \\
&\geq \prod_{i=0}^{n-3k-1} \left(1 - \frac{2k}{|V_i| - k}\right) \\
&= \prod_{i=0}^{n-3k-1} \left(1 - \frac{2k}{(n-i) - k}\right) \\
&= \prod_{i=0}^{n-3k-1} \left(\frac{n-i-3k}{n-i-k}\right) \\
&= \frac{n-3k}{n-k} \cdot \frac{n-1-3k}{n-1-k} \cdots \frac{n-5k}{n-3k} \cdots \frac{3k+1-3k}{3k+1-k} \\
&= \frac{n-3k}{n-k} \cdots \frac{1}{2k+1} = (2k)! \frac{(n-3k)!}{(n-k)!} \\
&\geq \Omega\left(\frac{1}{n^{2k}}\right),
\end{aligned}$$

the last bound as $n \gg k$ and $(2k)! = \Theta(1)$. Repeating this $O(n^{2k} \log(1/\delta))$ times and picking the best one suffices.

c) There are at most $O(n^{2k})$ unique values to return in step 2. Suppose every k-cut in $3k$ vertices are minimum, so we get at most $O(k^{3k} n^{2k}) = O(n^{2k})$.

d) There are at most $O(n^{2k})$ possible unique k-min-cuts.

Note that we can do better. Consider the same setting, at step i, we have V_i. We know that for any subset $S \subseteq V_i$ of size k, we have that

$$\sum_{v \in S} \deg(v) \geq |C|.$$

We can do something silly: make k copies of V_i, i.e., let $Y_j = V_i$ and $Y_1, \ldots, Y_k$. We have in total, $k \cdot |V_i|$ number of vertices' degree to sum over now; and we can rearrange such that all of them form distinct subsets of size k.

$$|E_i| = \frac{1}{2} \sum_{v \in V_i} \deg(v) = \frac{1}{2k} \sum_{v \in Y_1 \cup \cdots \cup Y_k} \deg(v) \geq \frac{1}{2k} |V_i||C|.$$

Redoing the probability calculation, we get

1	2	3	$\cdots$	k	$\cdots$	n
1	2	3	$\cdots$	k	$\cdots$	n
1	2	3	$\cdots$	k	$\cdots$	n
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$
1	2	3	$\cdots$	k	$\cdots$	n

Table 1: We can find one such arrangement through the diagonals.

$$\begin{aligned}
\Pr[C \text{ is preserved}] &= \prod_{i=0}^{n-2k-1} \left(1 - \frac{2k}{|V_i|}\right) \\
&\geq \prod_{i=0}^{n-2k-1} \left(\frac{n-i-2k}{n-i}\right) \\
&= \frac{n-2k}{n} \frac{n-1-2k}{n-1} \cdots \frac{n-4k}{n-2k} \cdots \frac{2k+1}{4k+1} \cdots \frac{1}{2k+1} \\
&= \frac{2k \cdot (2k-1) \cdots 1}{n \cdot (n-1) \cdots (n-2k+1)} \\
&= 1 / \binom{n}{2k} \\
&\geq \Omega\left(\frac{1}{n^{2k}}\right).
\end{aligned}$$

Problem 6. $(\star)$ Consider the following algorithm:

1. Draw, independently for every edge $e \in E$, a weight w_e in $[0,1]$ uniformly at random.
2. Build the MST of $G = (V, E, w)$ (according to these weights)
3. Remove the heaviest edge of the MST, and let A, B be the resulting 2 components.

4. Return (A, B) as cut.

Show that this is equivalent to Karger's algorithm (the "basic" version). Deduce how to implement this algorithm in time $O(m \log m)$. *Hint: Think about Kruskal's algorithm.*

Solution 6. The idea is to run Kruskal's algorithm: we will show it is equivalent to the contraction algorithm. First, by sampling the edge weights uniformly and independently in $[0, 1]$, sorting the edges by these weights gives us a uniformly random ordering of the edges. Moreover, this is still true at each time step t: even conditioned on having picked $t - 1$ edges, by independence of how we chose the weights, it is still true that the remaining edges are still permuted uniformly at random. Thus, when we go through the edges one by one in Kruskal, this is equivalent to sampling an edge u.a.r.

The step of Kruskal is then equivalent to a contraction:

- if that creates a cycle, this means the edge was already in a "supervertex" so we cannot contract it
- otherwise, we contract it. When we exclude edges (those edges are picked by MST algorithm), the rest is not affected (they still can be seen as having, based on their weights, a uniformly random ordering among themselves).

Thus, each time Kruskal's algorithm picks one edge in corresponds to a contraction, with the only difference that it contract *all* of them: so at the end we cut one edge to reverse that last contraction.

For the running time: (1) Go through all edges, generate a random number for each: $O(m)$ time. (2) Run Kruskal's algorithm: $O(m \log |V|) = O(m \log m)$ time.

Advanced

Problem 7. Prove that (a suitable modification of) Karger's algorithm still works for weighted graphs (with non-negative weights). Do the same for the Karger–Stein algorithm.

Solution 7. Pick e with $\Pr\left[\frac{w_e}{\sum_{e \in E} w_e}\right]$ instead. At each time $i = 0, \ldots, n-1$, V_i, E_i. Denote $W = \sum_{u \in A, v \in B} w(u, v)$, the weight of the min-cut. Let $C := \{(u, v) | u \in A, v \in B\}$.

The probability that it gets cut at step i, is

$$\Pr\left[\text{algo picks } e \in C\right] = \Pr\left[\frac{\sum_{e' \in C} w_{e'}}{\sum_{e \in E_i} w_e}\right].$$

The min weights around any one vertex is at least W, otherwise, contradiction. Then

$$\sum_{e \in E_i} w_e \geq \frac{1}{2}|V_i| \cdot W \Rightarrow \frac{\sum_{e' \in C} w_{e'}}{\sum_{e \in E_i} w_e} \leqslant \frac{2W}{|V_i| \cdot W} = \frac{2}{|V_i|}.$$

The rest follows the proof from the lecture notes (i.e., is similar to the unweighted case)

$$\Pr\left[C \text{ is returned}\right] \;=\; \prod_{i=0}^{n-2}\left(1-\Pr\left[\frac{\sum_{e'\in C} w_{e'}}{\sum_{e\in E_i} w_e}\right]\right)$$